

Combined Lesson: OpenClaw, Hermes & NanoClaw — Orchestration, Models, Edge, Heartbeats, Skills, Channels & A2A

Duration: ~7.5–8 hours (can split across three sessions: Session A = Parts 1–3, Session B = Parts 4–5 + lab, Session C = Parts 6–7 + final lab)

Audience: Working professionals / graduate learners (NUS-ISS format)

Learning Objectives

By the end of this session, learners will be able to:

1. Explain how OpenClaw's orchestration layer, Hermes as the model layer, and NanoClaw as the edge deployment layer fit together
2. Design a heartbeat/liveness mechanism for agents, including edge agents on unreliable networks
3. Write agent skills with descriptions that reliably trigger correct invocation, with basic security hardening
4. Wire an agent to real-world channels — messaging (Telegram/WhatsApp) and voice — and handle their distinct failure modes
5. Design a novel agent-to-agent (A2A) communication protocol on top of Redis Streams + MCP, distinct from off-the-shelf A2A implementations
6. Build an end-to-end flow: a user messages/speaks to an agent, the orchestrator dispatches to a healthy edge agent over the custom A2A protocol, which invokes a skill and replies back over the same channel

Session Outline

Time	Segment
0:00– 0:15	Opening & framing
0:15– 1:00	Part 1: OpenClaw – the orchestration layer
1:00– 1:45	Part 2: Hermes – the model layer
1:45– 1:55	Break
1:55– 2:40	Part 3: NanoClaw – the edge layer
2:40– 3:40	Part 4: Heartbeat Mechanisms (deep dive)
3:40– 3:50	Break
3:50– 4:50	Part 5: Skill Writing (deep dive)

Time	Segment
4:50– 5:00	Break
5:00– 6:00	Part 6: Channel Integration – Messaging & Voice
6:00– 7:00	Part 7: A2A Setup – a one-of-its- kind protocol
7:00– 7:20	Combined lab: end-to-end channel- to-skill flow over custom A2A
7:20– 7:30	Wrap-up & Q&A

(If splitting into three sessions: Session A ends after Part 3; Session B = Parts 4–5; Session C opens with a 5-min recap before Part 6, then covers Part 7 and the final lab.)

Opening & Framing (15 min)

- Three-layer framing: **orchestration**
(OpenClaw) → **model** (Hermes) → **edge**

deployment (NanoClaw)

- Two cross-cutting concerns that make the stack trustworthy in production: **heartbeats** (agent state) and **skill writing** (agent capability)
 - Preview: by the end, learners will build a small end-to-end system touching all five pieces
-

Part 1: OpenClaw – the Orchestration Layer (45 min)

- Multi-agent architecture: Redis Streams + MCP for agent-to-agent communication
 - Live demo: spin up two agents talking via Redis Streams
 - Where ClawHub (skill marketplace) and MoltGuard (guardrail proxy) plug into this layer
-

Part 2: Hermes – the Model Layer (45 min)

- Why an open-weight function-calling model matters for self-hosted/edge agents (cost, latency, no vendor lock-in)
- Compare Hermes' tool-calling format vs Claude/GPT function calling
- Hands-on: get Hermes making a structured tool call through an OpenClaw agent

Break (10 min)

Part 3: NanoClaw – the Edge Layer (45 min)

- Constraints of Raspberry Pi-class hardware: memory, quantization, latency
- Demo: deploy a slimmed-down OpenClaw + Hermes agent on NanoClaw
- Trade-offs vs cloud-hosted (AWS Bedrock/Claude) agents

Part 4: Heartbeat Mechanisms

— Deep Dive (60 min)

4.1 Concept (15 min)

- **Liveness vs readiness vs health** — three distinct signals often conflated:
 - *Liveness*: is the process running at all?
 - *Readiness*: can it currently accept work?
 - *Health*: is it running within acceptable performance/error bounds?
- **Push vs pull models**
 - Push: agent proactively pings orchestrator (lower orchestrator load, but agent must stay well-behaved)
 - Pull: orchestrator polls agent (simpler failure detection, scales worse with many agents)
- **Interval and TTL design**
 - Too frequent → noisy streams, wasted Redis throughput

- Too infrequent → slow failure detection, stale work assignment
- Rule of thumb: $TTL \approx 2-3 \times$ heartbeat interval, tuned to reassignment cost

4.2 OpenClaw-specific mechanics (15 min)

- Implementing heartbeat via a dedicated Redis Stream or a TTL'd key per agent
- Orchestrator-side detection loop: identifying a dead/unreachable agent and triggering reassignment
- **Flapping agents**: debounce logic — require N consecutive misses before declaring "down"; why single-miss detection causes thrashing

4.3 Edge case: NanoClaw agents (15 min)

- Intermittent connectivity on Pi-class hardware breaks naive heartbeat assumptions
- Backoff strategies: exponential backoff on reconnect attempts

- Last-known-state caching so the orchestrator doesn't prematurely kill/reassign work during a transient blip
- Trade-off: aggressive detection (fast reassignment, more false positives) vs lenient detection (fewer false positives, slower recovery)

4.4 Hands-on Lab (15 min)

1. Write an agent process that pushes a heartbeat entry every N seconds
2. Write an orchestrator loop that flags an agent "down" after missing M consecutive beats
3. Kill the agent process mid-session and measure detection time
4. Adjust N and M and re-test — discuss the resulting trade-off curve

Discussion prompt: What N/M values would you choose for a cloud-hosted agent vs a NanoClaw edge agent, and why?

Break (10 min)

Part 5: Skill Writing — Deep Dive (60 min)

5.1 Anatomy of a skill (15 min)

- Two halves: **metadata** (name, description, trigger conditions) and **execution logic**
- The description is not documentation — it *is* the interface the agent's tool-selection step reads

5.2 Writing for discoverability (15 min)

- Vague descriptions cause missed or misfired skill invocations
- Side-by-side: *"Handles data stuff"* (bad) vs *"Fetches current weather for a given city and date; use only when the user asks about weather conditions, not climate history"* (good)
- Exercise: learners critique 3–4 deliberately weak skill descriptions and rewrite them

5.3 Security angle (15 min)

- Ties into OWASP Agentic Applications guidance
- **Sandboxing execution:** isolate skill code so a malicious/buggy skill can't escalate privileges
- **Input validation:** validate agent-generated parameters before execution
- **Output validation:** sanitize skill output before it re-enters the agent's context/loop

5.4 Hands-on Lab (15 min)

1. Write a simple skill (e.g., unit converter) with a clear description
 2. Register it and confirm correct invocation on a matching prompt
 3. Vague-ify the description and re-test — observe misfire/no-invoke
 4. Restore the description, add basic input validation, try an adversarial input, confirm rejection
-

Part 6: Channel Integration — Messaging & Voice (60 min)

6.1 Messaging channels: Telegram & WhatsApp (20 min)

- Webhook vs long-polling ingestion — how an inbound message becomes an event on the OpenClaw Redis Stream
- Mapping a chat/session ID to an agent instance so multi-turn conversation state stays coherent
- Handling channel-specific quirks: message length limits, rich media (images, voice notes sent as files), delivery/read receipts
- Rate limits and retry/backoff when sending replies (both platforms throttle aggressively)
- Demo: inbound Telegram message → OpenClaw agent → skill invocation → reply back to the same chat

6.2 Voice channels (20 min)

- Two integration patterns: **turn-based** (record → transcribe → agent → synthesize → play) vs **streaming** (continuous ASR/TTS with barge-in support)
- Where ASR/TTS sits relative to Hermes: transcription happens before the model call, synthesis after — the agent itself still reasons over text
- Latency budget: why voice has a much tighter round-trip tolerance than messaging, and what that means for choosing cloud vs NanoClaw-local inference
- Handling interruptions/barge-in and silence detection — a UX problem, not just a technical one
- Demo (or walkthrough if no live mic setup): a short voice round-trip through the same OpenClaw agent used in the messaging demo

6.3 Channel abstraction in OpenClaw (10 min)

- Why channels should be a thin adapter layer, not baked into agent logic — the agent shouldn't need to know if it's talking to Telegram, WhatsApp, or a mic
- Normalizing inbound events (text, voice-transcribed-to-text, media) into one internal message format before they hit the Redis Stream
- Where this adapter layer intersects with heartbeats (Part 4): a channel adapter itself needs liveness tracking, since a dead webhook listener is as bad as a dead agent

6.4 Hands-on Lab (10 min)

1. Wire one messaging channel (Telegram or WhatsApp sandbox/test account) to a Part 1 OpenClaw agent
 2. Send a message that triggers the Part 5 skill; confirm the reply round-trips correctly
 3. Discuss: what would need to change to add a second channel without touching agent logic?
-

Part 7: A2A Setup – A One-of-its-Kind Protocol (60 min)

7.1 Why not just use an off-the-shelf A2A standard? (15 min)

- Quick survey of the emerging A2A landscape (e.g. agent-card discovery, task-based request/response protocols) and where they assume infrastructure
OpenClaw doesn't need (HTTP-centric, synchronous request/response)
- OpenClaw's differentiator: **Redis Streams as the transport**, giving durable, replayable, multi-consumer messaging for free – most A2A protocols bolt this on top of plain HTTP
- Design goal: an A2A layer that's natively stream-based, heartbeat-aware (Part 4), and channel-agnostic (Part 6), rather than a generic point-to-point RPC

7.2 Anatomy of the custom protocol (20 min)

- **Message envelope:** sender agent ID, recipient agent ID (or broadcast topic), task ID, payload, correlation ID for multi-turn task tracking
- **Stream topology:** one stream per agent inbox vs shared topic streams with consumer groups — trade-offs for fan-out (broadcast to multiple agents) vs fan-in (multiple agents reporting to one orchestrator)
- **Task lifecycle states:** proposed → accepted → in-progress → completed/failed, mirrored as stream entries so any consumer can reconstruct task history from the stream itself (replayability as a debugging feature)
- How this differs from typical A2A implementations: most treat "agent card" discovery and task negotiation as separate concerns from transport; here they're unified because the stream itself is the source of truth

7.3 Integrating heartbeats and skills into A2A (15 min)

- Reusing the Part 4 heartbeat stream as a *capability advertisement* channel – an agent's heartbeat payload can include which skills (Part 5) it currently has registered, so the orchestrator's routing decision uses one lookup instead of two
- Routing logic: given an incoming task, match required skill → find agents advertising that skill → filter to agents with a live heartbeat → dispatch
- Failure handling unique to this design: because tasks live on a replayable stream, a newly-spun-up agent (or a recovering NanoClaw device) can catch up on missed task assignments just by re-reading the stream from its last acknowledged offset

7.4 Hands-on Lab (10 min)

1. Define the message envelope schema for the custom A2A protocol
2. Implement a two-agent exchange: Agent A proposes a task, Agent B accepts and

reports progress via stream entries

3. Kill and restart Agent B mid-task; confirm it can resume from the stream rather than losing task state
 4. Discuss: what would break if you swapped this for a standard synchronous A2A protocol? What would you lose, and what would you gain?
-

Combined Lab: End-to-End Channel-to-Skill Flow over Custom A2A (20 min)

Goal: tie all seven parts together — a message (or voice input) arrives over a real channel, is routed via the custom A2A protocol to a NanoClaw edge agent running Hermes, which invokes a registered skill, while an OpenClaw orchestrator only dispatches to agents reporting healthy heartbeats and advertising the right capability.

1. Learners take their Part 6 channel-wired agent and confirm it's the same

NanoClaw+Hermes agent from Part 3, with the Part 5 skill, Part 4 heartbeat producer, and Part 7 A2A envelope handling already in place

2. Send a channel message that requires the skill to be invoked
3. Orchestrator uses the Part 7 routing logic (skill match + live heartbeat) to select and dispatch to the edge agent
4. Simulate the edge agent going down mid-task – confirm the orchestrator reroutes/queues, and that a recovering agent can resume from the stream rather than losing task state
5. Discuss: how does replayability change your answer to "what happens to an in-flight task" compared to the Part 4/5 discussion – and how does that surface back to the user over a messaging vs voice channel differently?

Wrap-up & Discussion (10 min)

- Recap the seven-layer story: orchestration (OpenClaw) → model (Hermes) → edge (NanoClaw) → trust in state (heartbeats) → trust in capability (skill writing) → trust in delivery (channels) → trust in coordination (custom A2A)
 - Open Q&A
 - Stretch topics for advanced learners: a combined health+capability registry so orchestrator routing uses one signal instead of two (foreshadowed in Part 7.3); streaming voice with barge-in as a follow-up deep dive; benchmarking the custom A2A protocol against a standard synchronous A2A implementation
-

Materials Checklist

- Redis instance (local or shared) for orchestration, heartbeat, and A2A stream labs
- NanoClaw device or Pi-class emulation environment

- Hermes model weights (quantized build suitable for edge demo)
 - Starter OpenClaw agent scaffold with Redis Streams + MCP pre-wired
 - Set of 3–4 deliberately weak skill descriptions for the critique exercise
 - Telegram and/or WhatsApp sandbox/test account with webhook access
 - Basic ASR/TTS setup (cloud API or local model) for the voice demo
 - Reference: OWASP Agentic Applications Cheat Sheet
 - Reference: overview of at least one standard A2A protocol spec (for the Part 7.1 comparison discussion)
-

Technical Appendix:

Configuring A2A + Hermes with LangGraph

This appendix gives the implementation-level detail for wiring Part 7's custom A2A protocol

together with Hermes, using LangGraph as the per-agent execution graph. Use this as instructor reference / handout for learners who want to build past the guided demo.

A.1 Why LangGraph fits here

- Each OpenClaw agent's internal reasoning loop (receive task → call Hermes → optionally invoke a skill/tool → respond) maps naturally onto a LangGraph `StateGraph` : nodes for "call model," "invoke skill," "await tool result," edges for conditional routing based on whether Hermes emitted a tool call
- LangGraph's built-in checkpointer concept lines up with the Part 7.2 task lifecycle (proposed → accepted → in-progress → completed/failed) — a checkpoint after each node execution gives the same replayability benefit as reading task state back from the Redis Stream

A.2 Wiring Hermes into a LangGraph node

- Hermes isn't a first-party LangChain/LangGraph integration, so serve it behind an OpenAI-compatible endpoint (e.g. vLLM or an Ollama/TGI server exposing `/v1/chat/completions`) and bind it via `ChatOpenAI`-style config pointed at that local endpoint
- Confirm Hermes' tool-calling output format matches what LangGraph's tool-calling node expects (OpenAI-style `tool_calls` in the response) — this was flagged as a comparison point back in Part 2, and it's the thing most likely to need a translation shim
- Bind the Part 5 skills to the graph as LangGraph tools; the tool's `description` field is exactly the skill description discussed in 5.2 — same discoverability rules apply

A.3 Connecting the graph to the Redis Streams A2A transport

- The graph itself handles a single agent's internal turn; the Part 7.2 message envelope (sender, recipient, task ID,

payload, correlation ID) becomes the **input state** that triggers a graph invocation and the **output state** that gets serialized back onto the outbound stream

- Pattern: a thin consumer loop reads new entries from the agent's inbox stream, maps each to a LangGraph input state, invokes the graph, and writes the resulting state back as a new stream entry (task status update or final response) – the graph doesn't need to know about Redis at all, keeping the two concerns cleanly separated
- Use LangGraph's checkpointer with a Redis-backed implementation (or a thin custom one) so an in-progress task's graph state survives an agent restart – this is what makes the Part 7.3 "recovering agent resumes from stream offset" behavior also resume mid-graph-execution, not just mid-task

A.4 Heartbeats and capability advertisement from inside the graph

- The graph doesn't own the heartbeat loop
 - keep heartbeat emission (Part 4) as a separate lightweight process alongside the graph consumer, not a graph node, since it needs to keep beating even while the graph is mid-execution on a long task
- That heartbeat process can introspect the graph's currently-bound tools to build the capability-advertisement payload described in 7.3, so skill registration stays a single source of truth (the graph's tool list) rather than a separately maintained config

A.5 Suggested demo/lab extension

1. Stand up Hermes behind an OpenAI-compatible local endpoint
2. Build a two-node LangGraph (model call → conditional tool invocation) using one Part 5 skill as a bound tool
3. Wrap it with the thin Redis Streams consumer/producer loop from A.3
4. Re-run the Part 7.4 kill-and-resume test, but this time confirm the LangGraph

checkpoint — not just the task ID — is what allows correct resumption

5. Compare latency/complexity against the framework-free version of the agent loop built in the main Part 7 lab